Day 1: Programming Fundamentals

Comprehensive Answer Key for the 50 Fundamental Questions (C / C++)

VARIABLES & NAMING CONVENTIONS

Question 1: What is a variable in C/C++?

A variable is a named storage location in the computer's memory (RAM) that holds a value. It acts as a container for data that can be manipulated, modified, and retrieved throughout the execution of a program.

Question 2: Why do we use variables in programs?

We use variables to store data dynamically, reuse values multiple times without re-entering them, label memory addresses with human-readable names, and allow programs to perform operations on changing inputs rather than hardcoded, static values.

Question 3: What is the difference between a variable name and its value?

- **Variable Name (Identifier):** The text label used in the source code to refer to the specific memory location (e.g., `age`).
- **Value:** The actual data stored inside that memory location at any given point in time (e.g., `18`).

Question 4: Which of the following is a valid variable name? [a) 2num b) total_marks c) for d) my marks]

b) total_marks is the valid variable name.

- `2num` is invalid because identifiers cannot start with a digit.
- `for` is invalid because it is a reserved keyword in C/C++.
- `my marks` is invalid because variable names cannot contain spaces.

Question 5: Why is Age different from age?

C and C++ are **case-sensitive** languages. This means the compiler treats uppercase and lowercase characters as completely unique, making `Age` and `age` two entirely distinct identifiers pointing to separate memory spaces.

Question 6: Can a variable name contain spaces? Why?

No, variable names cannot contain spaces. The compiler relies on whitespace (spaces, tabs, newlines) to separate distinct syntax elements, keywords, and identifiers. A space inside a name would cause a compilation error as the compiler tries to interpret it as two separate tokens.

Question 7: Can a variable name start with an underscore (_) ?

Yes, a variable name can start with an underscore. However, it is generally discouraged for general user variables because the standard library and system headers use leading underscores extensively for system-defined identifiers, which risks naming collisions.

Question 8: What happens if two variables have the same name in the same scope?

It causes a **"redeclaration error"** / **"redefinition error"** during compilation. The compiler will reject the program because it cannot distinguish which memory location you are referring to when that name is called.

Question 9: Write three meaningful variable names for storing student information.

1. `student_id` (or `studentId`)
2. `student_age` (or `studentAge`)
3. `gpa` (or `studentGpa`)

Question 10: Which naming convention is better and why: studentMarks or sm?

studentMarks is significantly better. It is descriptive and self-documenting, making the code highly readable and maintainable for humans. Conversely, `sm` is vague and ambiguous, which increases cognitive load and leads to bugs in larger projects.

DATA TYPES

Question 11: What is a data type?

A data type is an attribute that tells the compiler what kind of value a variable can store, how much memory space to allocate for it, and what operations can be validly performed on that data.

Question 12: Which data type is used to store whole numbers?

The `int` (integer) data type is used to store whole numbers (both positive and negative, without fractions).

Question 13: Which data type is used to store decimal values?

The `float` or `double` data types are used to store decimal/floating-point values.

Question 14: What is the difference between float and double?

- **float:** Single-precision floating-point. Usually takes 4 bytes of memory and provides about 6–7 digits of decimal precision.
- **double:** Double-precision floating-point. Usually takes 8 bytes of memory and provides about 15 digits of decimal precision, making it more accurate for complex math.

Question 15: Which data type stores a single character?

The `char` data type is used to store a single character, enclosed in single quotes (e.g., `'A'`).

Question 16: Which data type stores True/False values?

The `bool` (boolean) data type is used to store logical values: `true` or `false`.

Question 17: What data type is suitable for storing a person's name?

In C++, the `std::string` data type is ideal. In pure C, an array of characters (`char name[]` or `char*`) is used.

Question 18: Predict the output: int age=18; cout<<age;

Output: `18`

Question 19: Identify the data type: 25, 3.14, 'A', true.

- `25` → `int`
- `3.14` → `double` (or `float` if suffixed with 'f')
- `'A'` → `char`
- `true` → `bool`

Question 20: What happens if you store a decimal number in an int variable?

The decimal part is completely truncated (chopped off), not rounded. For example, if you store `9.99` in an `int`, the variable will strictly hold the value `9`.

INPUT & OUTPUT STREAMS

Question 21: What is the purpose of cin?

`cin` (character input) is the standard input stream object in C++. It is used to read data from the standard input device, which is typically the keyboard.

Question 22: What is the purpose of cout?

`cout` (character output) is the standard output stream object in C++. It is used to print and display text or variable values onto the standard output device, which is typically the console screen.

Question 23: Which symbol is used with cin?

The extraction (or stream get-from) operator: `>>`

Question 24: Which symbol is used with cout?

The insertion (or stream put-to) operator: `<<`

Question 25: Explain: cin >> age;

This statement pauses the program execution and waits for the user to type a value on the keyboard. Once entered, that input is extracted from the stream, converted to match the expected type, and stored into the variable named `age`.

Question 26: Explain: cout << age;

This statement takes the value currently stored inside the variable `age` and inserts it into the output stream, causing it to display on the console screen.

Question 27: What is the purpose of endl?

`endl` is a stream manipulator that performs two actions: it inserts a newline character (`' '`) into the output stream, and it flushes the output buffer immediately to guarantee text shows on-screen.

Question 28: Write a statement to input a student's marks.

```
cin >> studentMarks;
```

Question 29: Write a statement to display 'Welcome to C++'.

```
cout << "Welcome to C++";
```

Question 30: Predict the output: cout << 'Programming';

Compilation Error. In C++, single quotes are strictly for single characters (e.g., `'A'`). For string literals (multiple characters), double quotes must be used: `cout << "Programming";`.

OPERATORS

Question 31: What is an operator?

An operator is a special symbol that tells the compiler to perform a specific mathematical, logical, or relational manipulation on data values (operands).

Question 32: Which operator is used for addition?

The plus symbol: `+`

Question 33: What is the result of 10 % 3?

Output: `1` (The modulo operator `%` returns the remainder of the integer division. 10 divided by 3 leaves a remainder of 1).

Question 34: What is the result of 8 % 2?

Output: `0` (Since 8 is perfectly divisible by 2, there is no remainder).

Question 35: Which operator checks equality?

The double equals operator: `==`

Question 36: What is the difference between = and == ?

- `=` is the **assignment operator**. It copies the value from its right-hand side into the variable on its left-hand side.
- `==` is the **relational equality operator**. It compares the values on both sides and returns `true` if they match, otherwise `false`.

Question 37: What is the result of 5 > 3?

Output: `true` (or `1` in integral contexts), since 5 is numerically greater than 3.

Question 38: What does && represent?

The **Logical AND** operator. It evaluates to `true` only if both conditions/operands on its left and right sides are true.

Question 39: What does || represent?

The **Logical OR** operator. It evaluates to `true` if at least one of the conditions/operands on either side is true.

Question 40: What does ! represent?

The **Logical NOT** (Inversion) operator. It reverses the logical state of its operand (turns `true` to `false`, and `false` to `true`).

CODING PROGRAMS & LOGIC

Question 41: Write a program to print 'Hello World'.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World" << endl;
    return 0;
}
```

Question 42: Write a program to input two numbers and display their sum.

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2, sum;
    cout << "Enter two integers: ";
    cin >> num1 >> num2;
    sum = num1 + num2;
    cout << "The sum is: " << sum << endl;
    return 0;
}
```

Question 43: Write a program to calculate the area of a rectangle.

```
#include <iostream>
using namespace std;

int main() {
    double length, width, area;
    cout << "Enter length and width of rectangle: ";
    cin >> length >> width;
    area = length * width;
    cout << "Area of rectangle = " << area << endl;
    return 0;
}
```

Question 44: What formula is used to calculate Simple Interest?

The formula is: $SI = (P \times R \times T) / 100$

Where: P = Principal amount, R = Rate of interest per annum, and T = Time period in years.

Question 45: Write a program to calculate Simple Interest.

```
#include <iostream>
using namespace std;

int main() {
    double principal, rate, time, simple_interest;
    cout << "Enter Principal, Rate of Interest, and Time (years): ";
    cin >> principal >> rate >> time;
    simple_interest = (principal * rate * time) / 100.0;
    cout << "Simple Interest = " << simple_interest << endl;
    return 0;
}
```

Question 46: Why is a temporary variable used while swapping two numbers?

If you assign `a = b;` directly, the original value of `a` is immediately overwritten and lost. A third temporary variable acts as a backup storage box to hold the value of `a` safe while `b` overwrites it, allowing the value to be assigned to `b` afterward.

Question 47: Swap the values: a=10, b=20.

Logic using a temporary variable:

```
int a = 10, b = 20;
int temp;

temp = a;  // temp becomes 10
a = b;     // a becomes 20
b = temp;  // b becomes 10
```

Question 48: What formula is used to convert Celsius into Fahrenheit?

The formula is: $F = (C \times 9/5) + 32$ or $F = (C \times 1.8) + 32$

Question 49: What is the Fahrenheit value of 0°C?

Using the formula: $F = (0 \times 1.8) + 32 = 32$.
Therefore, 0°C is equal to **32°F**.

Question 50: Why can integer division give incorrect results in the Celsius-to-Fahrenheit program?

In C/C++, evaluating the fraction `9 / 5` using literal integers triggers **integer division**, which discards fractions and yields exactly `1` instead of `1.8`. This introduces massive calculation error. To resolve this, you must force floating-point division by writing it with decimals as `9.0 / 5.0` or `9.0 / 5`.